

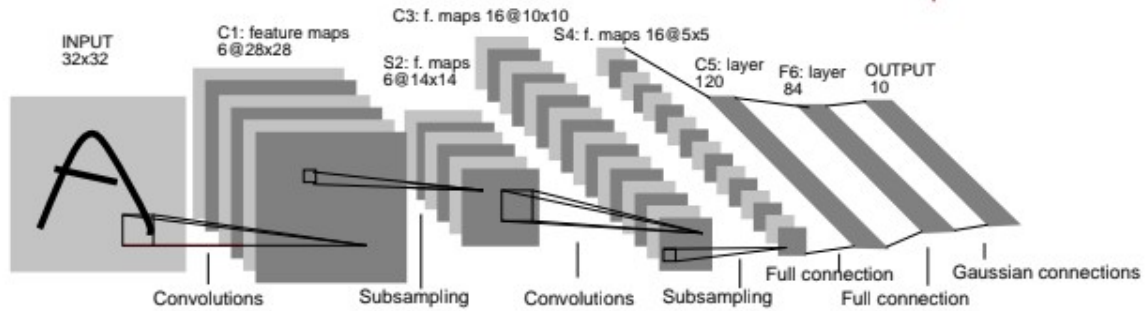


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

as the feature maps in the previous layer. The trainable coefficient and bias control the effect of the sigmoid non-linearity. If the coefficient is small, then the unit operates in a quasi-linear mode, and the sub-sampling layer merely blurs the input. If the coefficient is large, sub-sampling units can be seen as performing a “noisy OR” or a “noisy AND” function depending on the value of the bias. Successive layers of convolutions and sub-sampling are typically alternated, resulting in a “bi-pyramid”: at each layer, the number of feature maps is increased as the spatial resolution is decreased. Each unit in the third hidden layer in figure 2 may have input connections from several feature maps in the previous layer. The convolution/sub-sampling combination, inspired by Hubel and Wiesel’s notions of “simple” and “complex” cells, was implemented in Fukushima’s Neocognitron [32], though no globally supervised learning procedure such as back-propagation was available then. A large degree of invariance to geometric transformations of the input can be achieved with this progressive reduction of spatial resolution compensated by a progressive increase of the richness of the representation (the number of feature maps).

Since all the weights are learned with back-propagation, convolutional networks can be seen as synthesizing their own feature extractor. The weight sharing technique has the interesting side effect of reducing the number of free parameters, thereby reducing the “capacity” of the machine and reducing the gap between test error and training error [34]. The network in figure 2 contains 340,908 connections, but only 60,000 trainable free parameters because of the weight sharing.

Fixed-size Convolutional Networks have been applied to many applications, among other handwriting recognition [35], [36], machine-printed character recognition [37], on-line handwriting recognition [38], and face recognition [39]. Fixed-size convolutional networks that share weights along a single temporal dimension are known as Time-Delay Neural Networks (TDNNs). TDNNs have been used in phoneme recognition (without sub-sampling) [40], [41], spoken word recognition (with sub-sampling) [42], [43], on-line recognition of isolated handwritten characters [44], and signature verification [45].

### B. LeNet-5

This section describes in more detail the architecture of LeNet-5, the Convolutional Neural Network used in the experiments. LeNet-5 comprises 7 layers, not counting the input, all of which contain trainable parameters (weights). The input is a 32x32 pixel image. This is significantly larger than the largest character in the database (at most 20x20 pixels centered in a 28x28 field). The reason is that it is desirable that potential distinctive features such as stroke end-points or corner can appear in the center of the receptive field of the highest-level feature detectors. In LeNet-5 the set of centers of the receptive fields of the last convolutional layer (C3, see below) form a 20x20 area in the center of the 32x32 input. The values of the input pixels are normalized so that the background level (white) corresponds to a value of -0.1 and the foreground (black) corresponds to 1.175. This makes the mean input roughly 0, and the variance roughly 1 which accelerates learning [46].

In the following, convolutional layers are labeled  $C_x$ , sub-sampling layers are labeled  $S_x$ , and fully-connected layers are labeled  $F_x$ , where  $x$  is the layer index.

Layer C1 is a convolutional layer with 6 feature maps. Each unit in each feature map is connected to a 5x5 neighborhood in the input. The size of the feature maps is 28x28 which prevents connection from the input from falling off the boundary. C1 contains 156 trainable parameters, and 122,304 connections.

Layer S2 is a sub-sampling layer with 6 feature maps of size 14x14. Each unit in each feature map is connected to a 2x2 neighborhood in the corresponding feature map in C1. The four inputs to a unit in S2 are added, then multiplied by a trainable coefficient, and added to a trainable bias. The result is passed through a sigmoidal function. The 2x2 receptive fields are non-overlapping, therefore feature maps in S2 have half the number of rows and column as feature maps in C1. Layer S2 has 12 trainable parameters and 5,880 connections.

Layer C3 is a convolutional layer with 16 feature maps. Each unit in each feature map is connected to several 5x5 neighborhoods at identical locations in a subset of S2’s feature maps. Table I shows the set of S2 feature maps

trainable  
sub sample  
ie.  
 $z = \text{avg}(x_i) + b$



|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
|---|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|
| 0 | X |   |   |   |   | X | X | X |   | X | X  | X  | X  |    | X  | X  |
| 1 | X | X |   |   |   |   | X | X | X |   | X  | X  | X  | X  |    | X  |
| 2 | X | X | X |   |   |   | X | X | X |   | X  |    | X  | X  | X  | X  |
| 3 | X | X | X | X | X |   | X | X | X | X |    |    | X  |    | X  | X  |
| 4 |   |   | X | X | X |   |   | X | X | X | X  |    | X  | X  |    | X  |
| 5 |   |   |   | X | X | X |   |   | X | X | X  | X  |    | X  | X  | X  |

TABLE I

EACH COLUMN INDICATES WHICH FEATURE MAP IN S2 ARE COMBINED BY THE UNITS IN A PARTICULAR FEATURE MAP OF C3.

Partial connectivity

combined by each C3 feature map. Why not connect every S2 feature map to every C3 feature map? The reason is twofold. First, a non-complete connection scheme keeps the number of connections within reasonable bounds. More importantly, it forces a break of symmetry in the network. Different feature maps are forced to extract different (hopefully complementary) features because they get different sets of inputs. The rationale behind the connection scheme in table I is the following. The first six C3 feature maps take inputs from every contiguous subsets of three feature maps in S2. The next six take input from every contiguous subset of four. The next three take input from some discontinuous subsets of four. Finally the last one takes input from all S2 feature maps. Layer C3 has 1,516 trainable parameters and 151,600 connections.

Layer S4 is a sub-sampling layer with 16 feature maps of size 5x5. Each unit in each feature map is connected to a 2x2 neighborhood in the corresponding feature map in C3, in a similar way as C1 and S2. Layer S4 has 32 trainable parameters and 2,000 connections.

Layer C5 is a convolutional layer with 120 feature maps. Each unit is connected to a 5x5 neighborhood on all 16 of S4's feature maps. Here, because the size of S4 is also 5x5, the size of C5's feature maps is 1x1: this amounts to a full connection between S4 and C5. C5 is labeled as a convolutional layer, instead of a fully-connected layer, because if LeNet-5 input were made bigger with everything else kept constant, the feature map dimension would be larger than 1x1. This process of dynamically increasing the size of a convolutional network is described in the section Section VII. Layer C5 has 48,120 trainable connections.

Layer F6 contains 84 units (the reason for this number comes from the design of the output layer, explained below) and is fully connected to C5. It has 10,164 trainable parameters.

As in classical neural networks, units in layers up to F6 compute a dot product between their input vector and their weight vector, to which a bias is added. This weighted sum, denoted  $a_i$  for unit  $i$ , is then passed through a sigmoid squashing function to produce the state of unit  $i$ , denoted by  $x_i$ :

$$x_i = f(a_i) \quad (5)$$

The squashing function is a scaled hyperbolic tangent:

$$f(a) = A \tanh(Sa) \quad (6)$$

where  $A$  is the amplitude of the function and  $S$  determines its slope at the origin. The function  $f$  is odd, with horizontal asymptotes at  $+A$  and  $-A$ . The constant  $A$  is chosen to be 1.7159. The rationale for this choice of a squashing function is given in Appendix A.

Finally, the output layer is composed of Euclidean Radial Basis Function units (RBF), one for each class, with 84 inputs each. The outputs of each RBF unit  $y_i$  is computed as follows:

$$y_i = \sum_j (x_j - w_{ij})^2 \quad (7)$$

In other words, each output RBF unit computes the Euclidean distance between its input vector and its parameter vector. The further away is the input from the parameter vector, the larger is the RBF output. The output of a particular RBF can be interpreted as a penalty term measuring the fit between the input pattern and a model of the class associated with the RBF. In probabilistic terms, the RBF output can be interpreted as the unnormalized negative log-likelihood of a Gaussian distribution in the space of configurations of layer F6. Given an input pattern, the loss function should be designed so as to get the configuration of F6 as close as possible to the parameter vector of the RBF that corresponds to the pattern's desired class. The parameter vectors of these units were chosen by hand and kept fixed (at least initially). The components of those parameters vectors were set to -1 or +1. While they could have been chosen at random with equal probabilities for -1 and +1, or even chosen to form an error correcting code as suggested by [47], they were instead designed to represent a stylized image of the corresponding character class drawn on a 7x12 bitmap (hence the number 84). Such a representation is not particularly useful for recognizing isolated digits, but it is quite useful for recognizing strings of characters taken from the full printable ASCII set. The rationale is that characters that are similar, and therefore confusable, such as uppercase O, lowercase o, and zero, or lowercase l, digit 1, square brackets, and uppercase I, will have similar output codes. This is particularly useful if the system is combined with a linguistic post-processor that can correct such confusions. Because the codes for confusable classes are similar, the output of the corresponding RBFs for an ambiguous character will be similar, and the post-processor will be able to pick the appropriate interpretation. Figure 3 gives the output codes for the full ASCII set.

Another reason for using such distributed codes, rather than the more common "1 of N" code (also called place code, or grand-mother cell code) for the outputs is that non distributed codes tend to behave badly when the number of classes is larger than a few dozens. The reason is that output units in a non-distributed code must be off most of the time. This is quite difficult to achieve with sigmoid units. Yet another reason is that the classifiers are often used to not only recognize characters, but also to reject non-characters. RBFs with distributed codes are more appropriate for that purpose because unlike sigmoids, they are activated within a well circumscribed region of their in-

Notes :

! Architecture :

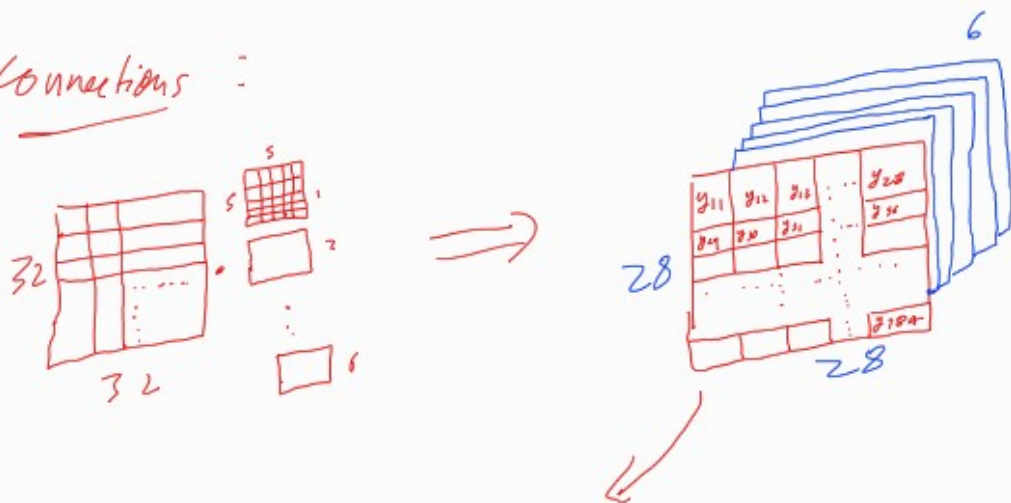
I/P:  $32 \times 32$

C<sub>1</sub>:  $5 \times 5 \times 6 \Rightarrow \underbrace{28 \times 28 \times 6}_{\text{o/p}}$   
filter channels

Parameters =  $\left( \begin{array}{c} \text{5x5 grid} \\ \text{channel 1} \end{array} + 1 \text{ bias} \right) \times 6 \text{ channels}$

$$\underbrace{(5 \times 5) \times 6}_w + \underbrace{1 \times 6}_b = 156$$

Connections :



$$y_{11} = (w_1 x_1 + w_2 x_2 + w_3 x_3 + \dots + w_{25} x_{25}) + b_1$$

i.e. 1 output cell = 25 weights connections + 1 bias connection

$$\therefore \underbrace{784 \text{ output cells}}_{(28 \times 28)} = (25 \times 784) + 784$$

by 1 filter channel

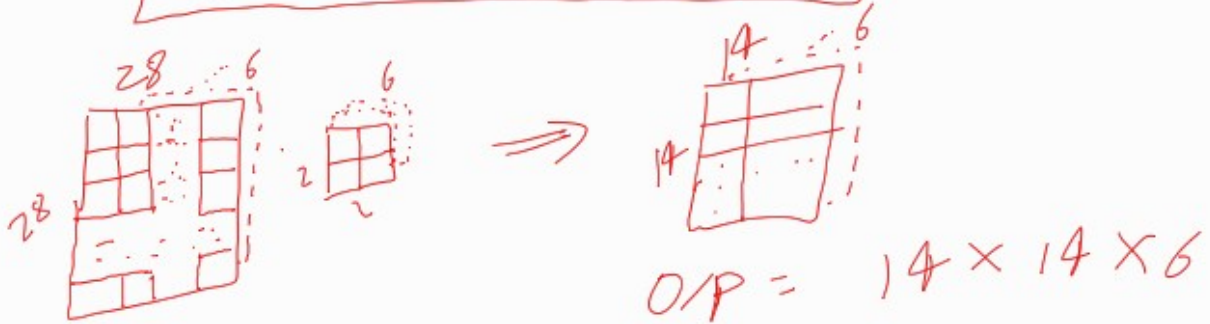
$$\therefore \text{For } \underline{6 \text{ channels}} = (25 \times 784) \times 6 + 784 \times 6 = \boxed{122304 \text{ connections}}$$



S2:

As per paper,

$$x_{ss} = \sigma(a \cdot \sum x + b) \quad \text{--- (1)}$$



Parameters = 2/channel (a & b) [From (1)]

$\therefore$  For 6 channels  $\Rightarrow$  12 parameters

Connections =

$y_{11} = \sigma(a \cdot (x_{11} + x_{12} + x_{21} + x_{22}) + b)$   
i.e.

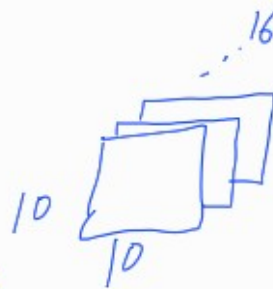
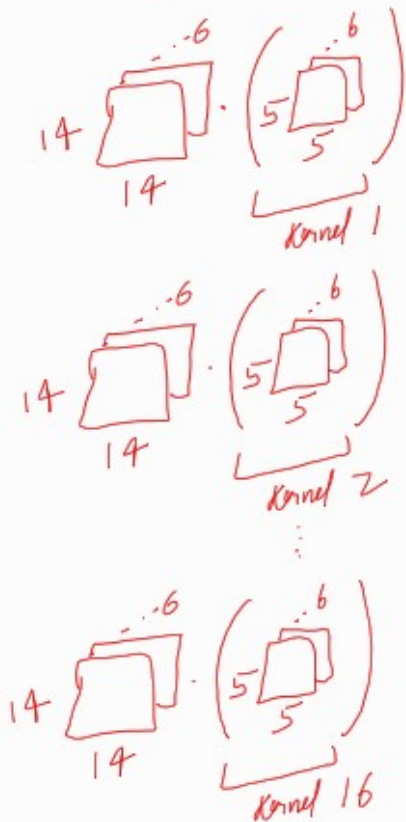
$y_{11} = \begin{bmatrix} a & a \\ a & a \end{bmatrix} \cdot \begin{bmatrix} x_1 & x_2 \\ x_3 & x_4 \end{bmatrix} + b$   
4 + 1 connections

$\therefore$  For  $14 \times 14 \times 6 = 14 \times 14 \times 6 \times (4 + 1)$   
= 5880 connections

\* → In modern CNN layers we use pooling layers instead of sub sampling layers.

L3

Ideally,



i.e.

$$g_{111} = w_{111} \cdot x_{111} + \dots + w_{556} \cdot x_{556}$$

i.e.

For 1 output cell,

$W = 5 \times 5 \times 6$  weight connection  
& 1 bias connection

For  $10 \times 10$  output cells,  
connections =  $(5 \times 5 \times 6) \times 100 + 1 \times 10$

For 16 channels,

$$\begin{aligned} \text{Connections} &= (5 \times 5 \times 6) \times 100 \times 6 + 16 \times 10 \\ &= 241600 \\ &\quad \text{connections} \end{aligned}$$

[∵ element wise multiplication gives 1 scalar i.e.

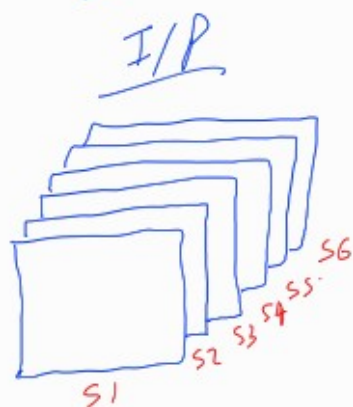
$$\sum_{i,j,k} x_{i,j,k} \cdot w_{i,j,k} = \boxed{\text{1 scalar}}$$

Parameters =

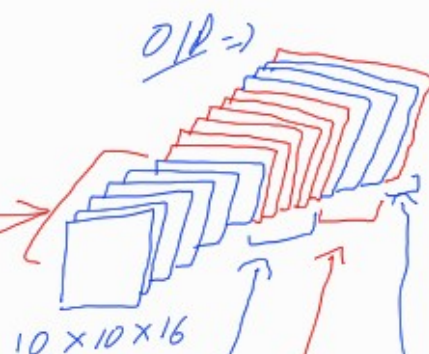
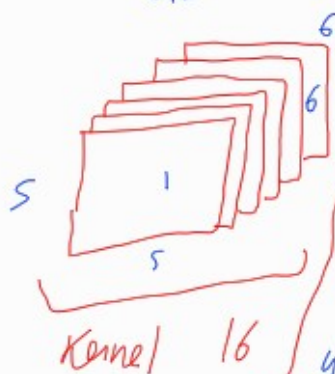
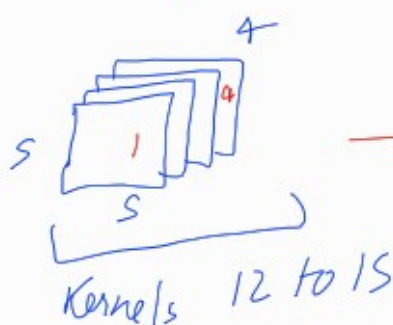
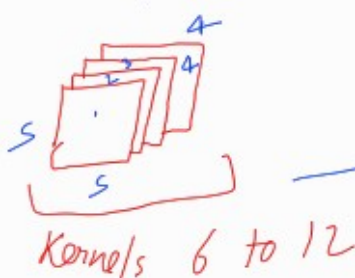
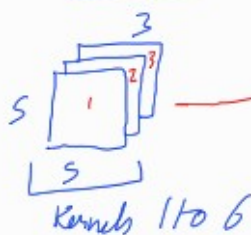
$$\begin{aligned} &(5 \times 5 \times 6) \times 16 + 16 \times 1 \\ &= \boxed{2416} \end{aligned}$$

But this wasn't done by LeCun.

Instead -



Filters/Kernel



Connections

Group 1 (1 to 6):

$$y_{111} = w_{111} \cdot x_{111} + \dots + w_{5,5,3} \cdot x_{5,5,3} + b$$

Parameters:

$$6 \times (5 \times 5 \times 3) + 6 = 456$$

$$6 \times (5 \times 5 \times 4) + 6 = 606$$

$$3 \times (5 \times 5 \times 4) + 3 = 303$$

$$(5 \times 5 \times 6) + 1 = 151$$

$$\underline{\underline{1516}}$$

$\therefore$  For 1 cell  $\Rightarrow$  75 w connections & 1 bias conn.

$\therefore$  For 10x10 cells  $\Rightarrow$  7500 w conn. & 100 bias conn.

$\therefore$  For 6 channels  $\Rightarrow$

$$7500 \times 6 + 100 \times 6$$

$$= 45600 \text{ connections.}$$

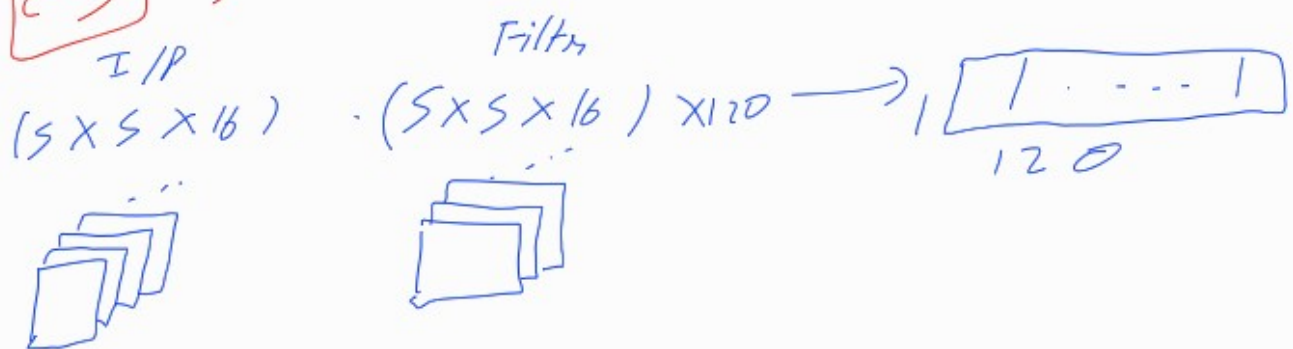
Similarly for all,

$$\underline{\underline{151600 \text{ connections}}}$$



(S4)  $\rightarrow$  Similar to S2

(S5)  $\rightarrow$



For channel 1  $\rightarrow$

$$y_{1,1,1} = (w_{1,1,1} \cdot x_{1,1,1} + \dots + w_{5,5,16} \cdot x_{5,5,16}) + b = 1 \text{ scalar} \quad \text{--- ①}$$

Similarly for 120 channels  $\rightarrow$

120 scalars  $\Rightarrow$  1 vector / tensor.

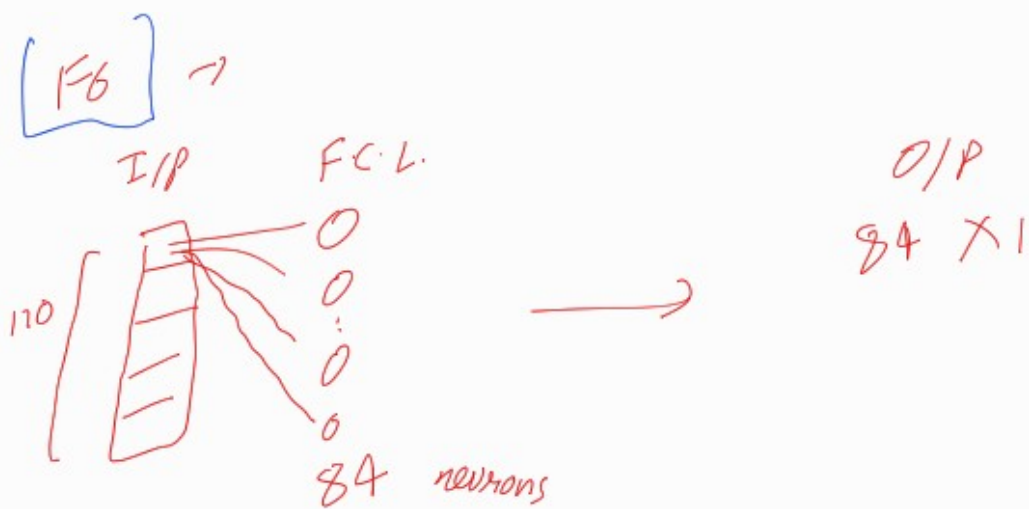
Parameters =

$$(5 \times 5 \times 16) \times 120 + 120 \times 1 = \underline{\underline{48120}}$$

Connections =

1 unit = 400 weight connections + 1 bias connection

$$\therefore 120 \text{ units} = 401 \times 120 = \underline{\underline{48120 \text{ connections}}}$$



Parameters =

$$y_i = \sum_{i=1}^{120} w_i \cdot x_i + b$$

$\Rightarrow y_i$  has 120  $w$  & 1  $b$

$\therefore$  84 neurons  $\Rightarrow$

$$84 \times 120 + 84 = \underline{\underline{10164}}$$

Squashing function (Activation function) 2

$\because$  A neuron outputs,  $(A_i = \sum_j w_j x_j + b)$ ,  $A_i$  could be anything like  $\{-100, 17, 0.5, 1000\}$ ; hence we apply 'tanh' based activation, such that

$$x_i = f(A_i) \Rightarrow \because \boxed{-1.7159 \leq x_i \leq 1.7159}$$

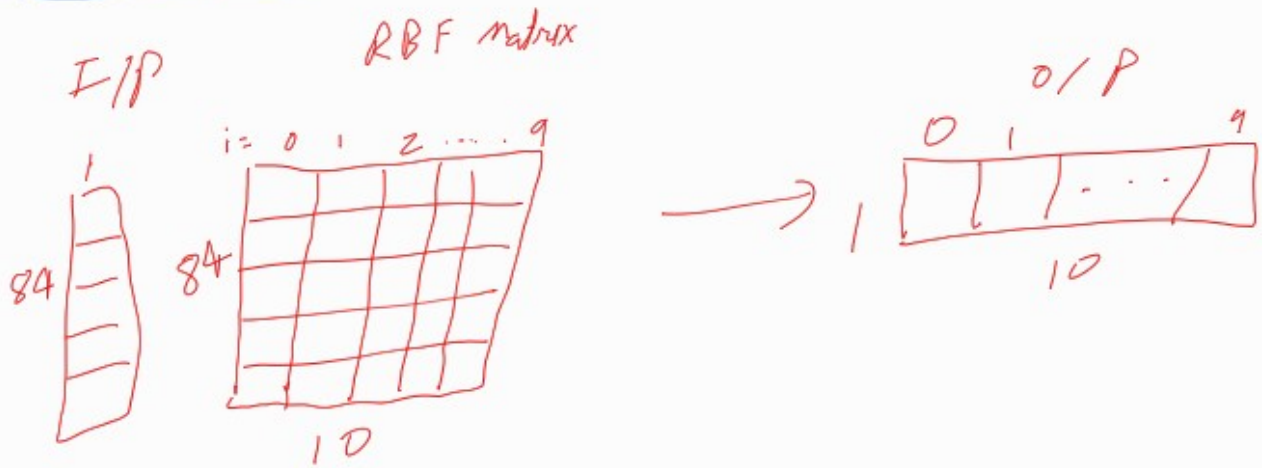
$\Downarrow$

here  $f(a) = 1.7159 \cdot \tanh\left(\frac{z}{3} \cdot a\right)$



Squashing applied after each layer upto F6.

Output layer :



$$x = (x_1, x_2, \dots, x_{84})$$

$$w_i = (w_{i1}, w_{i2}, \dots, w_{i84})$$

$$i \in \{0, \dots, 9\}$$

For each  $i$ ,

$$y_i = \sum_{j=1}^{84} (x_j - w_{ij})^2$$

$\therefore y_0 =$  distance from digit 0

$y_1 =$  " " " "

$\vdots$



Fig. 3. Initial parameters of the output RBFs for recognizing the full ASCII set.

put space that non-typical patterns are more likely to fall outside of.

The parameter vectors of the RBFs play the role of target vectors for layer F6. It is worth pointing out that the components of those vectors are +1 or -1, which is well within the range of the sigmoid of F6, and therefore prevents those sigmoids from getting saturated. In fact, +1 and -1 are the points of maximum curvature of the sigmoids. This forces the F6 units to operate in their maximally non-linear range. Saturation of the sigmoids must be avoided because it is known to lead to slow convergence and ill-conditioning of the loss function.

### C. Loss Function

The simplest output loss function that can be used with the above network is the Maximum Likelihood Estimation criterion (MLE), which in our case is equivalent to the Minimum Mean Squared Error (MSE). The criterion for a set of training samples is simply:

$$E(W) = \frac{1}{P} \sum_{p=1}^P y_{D^p}(Z^p, W) \quad (8)$$

where  $y_{D^p}$  is the output of the  $D^p$ -th RBF unit, i.e. the one that corresponds to the correct class of input pattern  $Z^p$ . While this cost function is appropriate for most cases, it lacks three important properties. First, if we allow the parameters of the RBF to adapt,  $E(W)$  has a trivial, but totally unacceptable, solution. In this solution, all the RBF parameter vectors are equal, and the state of F6 is constant and equal to that parameter vector. In this case the network happily ignores the input, and all the RBF outputs are equal to zero. This collapsing phenomenon does not occur if the RBF weights are not allowed to adapt. The second problem is that there is no competition between the classes. Such a competition can be obtained by using a more discriminative training criterion, dubbed the MAP (maximum a posteriori) criterion, similar to Maximum Mutual Information criterion sometimes used to train HMMs [48], [49], [50]. It corresponds to maximizing the posterior probability of the correct class  $D^p$  (or minimizing the logarithm of the probability of the correct class), given that the input image can come from one of the classes or from a background "rubbish" class label. In terms of

penalties, it means that in addition to pushing down the penalty of the correct class like the MSE criterion, this criterion also pulls up the penalties of the incorrect classes:

$$E(W) = \frac{1}{P} \sum_{p=1}^P (y_{D^p}(Z^p, W) + \log(e^{-j} + \sum_i e^{-y_i(Z^p, W)})) \quad (9)$$

The negative of the second term plays a "competitive" role. It is necessarily smaller than (or equal to) the first term, therefore this loss function is positive. The constant  $j$  is positive, and prevents the penalties of classes that are already very large from being pushed further up. The posterior probability of this rubbish class label would be the ratio of  $e^{-j}$  and  $e^{-j} + \sum_i e^{-y_i(Z^p, W)}$ . This discriminative criterion prevents the previously mentioned "collapsing effect" when the RBF parameters are learned because it keeps the RBF centers apart from each other. In Section VI, we present a generalization of this criterion for systems that learn to classify multiple objects in the input (e.g., characters in words or in documents).

Computing the gradient of the loss function with respect to all the weights in all the layers of the convolutional network is done with back-propagation. The standard algorithm must be slightly modified to take account of the weight sharing. An easy way to implement it is to first compute the partial derivatives of the loss function with respect to each connection, as if the network were a conventional multi-layer network without weight sharing. Then the partial derivatives of all the connections that share a same parameter are added to form the derivative with respect to that parameter.

Such a large architecture can be trained very efficiently, but doing so requires the use of a few techniques that are described in the appendix. Section A of the appendix describes details such as the particular sigmoid used, and the weight initialization. Section B and C describe the minimization procedure used, which is a stochastic version of a diagonal approximation to the Levenberg-Marquardt procedure.

## III. RESULTS AND COMPARISON WITH OTHER METHODS

While recognizing individual digits is only one of many problems involved in designing a practical recognition system, it is an excellent benchmark for comparing shape recognition methods. Though many existing method combine a hand-crafted feature extractor and a trainable classifier, this study concentrates on adaptive methods that operate directly on size-normalized images.

### A. Database: the Modified NIST set

The database used to train and test the systems described in this paper was constructed from the NIST's Special Database 3 and Special Database 1 containing binary images of handwritten digits. NIST originally designated SD-3 as their training set and SD-1 as their test set. However, SD-3 is much cleaner and easier to recognize than SD-1. The reason for this can be found on the fact that SD-3

↪  $e^{-j}$



was collected among Census Bureau employees, while SD-1 was collected among high-school students. Drawing sensible conclusions from learning experiments requires that the result be independent of the choice of training set and test among the complete set of samples. Therefore it was necessary to build a new database by mixing NIST's datasets.

SD-1 contains 58,527 digit images written by 500 different writers. In contrast to SD-3, where blocks of data from each writer appeared in sequence, the data in SD-1 is scrambled. Writer identities for SD-1 are available and we used this information to unscramble the writers. We then split SD-1 in two: characters written by the first 250 writers went into our new training set. The remaining 250 writers were placed in our test set. Thus we had two sets with nearly 30,000 examples each. The new training set was completed with enough examples from SD-3, starting at pattern # 0, to make a full set of 60,000 training patterns. Similarly, the new test set was completed with SD-3 examples starting at pattern # 35,000 to make a full set with 60,000 test patterns. In the experiments described here, we only used a subset of 10,000 test images (5,000 from SD-1 and 5,000 from SD-3), but we used the full 60,000 training samples. The resulting database was called the Modified NIST, or MNIST, dataset.

The original black and white (bilevel) images were size normalized to fit in a 20x20 pixel box while preserving their aspect ratio. The resulting images contain grey levels as result of the anti-aliasing (image interpolation) technique used by the normalization algorithm. Three versions of the database were used. In the first version, the images were centered in a 28x28 image by computing the center of mass of the pixels, and translating the image so as to position this point at the center of the 28x28 field. In some instances, this 28x28 field was extended to 32x32 with background pixels. This version of the database will be referred to as the *regular* database. In the second version of the database, the character images were deslanted and cropped down to 20x20 pixels images. The deslanting computes the second moments of inertia of the pixels (counting a foreground pixel as 1 and a background pixel as 0), and shears the image by horizontally shifting the lines so that the principal axis is vertical. This version of the database will be referred to as the *deslanted* database. In the third version of the database, used in some early experiments, the images were reduced to 16x16 pixels. The regular database (60,000 training examples, 10,000 test examples size-normalized to 20x20, and centered by center of mass in 28x28 fields) is available at <http://www.research.att.com/yann/ocr/mnist>. Figure 4 shows examples randomly picked from the test set.

### B. Results

Several versions of LeNet-5 were trained on the regular MNIST database. 20 iterations through the entire training data were performed for each session. The values of the global learning rate  $\eta$  (see Equation 21 in Appendix C for a definition) was decreased using the following schedule: 0.0005 for the first two passes, 0.0002 for the next



Fig. 4. Size-normalized examples from the MNIST database.

three, 0.0001 for the next three, 0.00005 for the next 4, and 0.00001 thereafter. Before each iteration, the diagonal Hessian approximation was reevaluated on 500 samples, as described in Appendix C and kept fixed during the entire iteration. The parameter  $\mu$  was set to 0.02. The resulting effective learning rates during the first pass varied between approximately  $7 \times 10^{-5}$  and 0.016 over the set of parameters. The test error rate stabilizes after around 10 passes through the training set at 0.95%. The error rate on the training set reaches 0.35% after 19 passes. Many authors have reported observing the common phenomenon of over-training when training neural networks or other adaptive algorithms on various tasks. When over-training occurs, the training error keeps decreasing over time, but the test error goes through a minimum and starts increasing after a certain number of iterations. While this phenomenon is very common, it was not observed in our case as the learning curves in figure 5 show. A possible reason is that the learning rate was kept relatively large. The effect of this is that the weights never settle down in the local minimum but keep oscillating randomly. Because of those fluctuations, the average cost will be lower in a broader minimum. Therefore, stochastic gradient will have a similar effect as a regularization term that favors broader minima. Broader minima correspond to solutions with large entropy of the parameter distribution, which is beneficial to the generalization error.

The influence of the training set size was measured by training the network with 15,000, 30,000, and 60,000 examples. The resulting training error and test error are shown in figure 6. It is clear that, even with specialized architectures such as LeNet-5, more training data would improve the accuracy.

To verify this hypothesis, we artificially generated more training examples by randomly distorting the original training images. The increased training set was composed of the 60,000 original patterns plus 540,000 instances of



## Implementation flow :

Leaf to roots

1. Scaled Tanh class  $\rightarrow$  used in every other layer  
 $\rightarrow$  implement the formulae
2. Sub sampling class  $\rightarrow$  S2 & S4
3. C3 table & C3 sparse layer
4. Euclidean RBF
5. LeNet class
6. loss